Cahier des Charges — Akinator Robot

Oct 28, 2026 · @jk

1. Context and objectives

The club will build an interactive "Akinator" robot: a physical device with a screen, microphone and camera that guesses what a player is thinking of by asking yes/no questions, powered by a backend running face detection and a large language model (LLM). The same game is also playable from a web interface.

Goals

- Deliver a working robot that plays a full game (greeting → 5–20 questions → guess) by voice and buttons.
- Detect and greet players by face, and track one player per session.
- Use an LLM to choose the next question and decide when to guess.
- Offer a web version sharing the same backend and game logic.
- Give club members hands-on experience in embedded systems, AI, backend and UI/UX design.

Out of scope (v1): recognizing identities by name (face *recognition*), multiplayer games, and fully offline LLM operation.

2. Scope, users and use cases

The system serves three kinds of users across two front ends (robot and web).

User	Where	Main needs
Player (visitor, student)	Robot or web	Start a game quickly, answer by voice or button, see a fun guess
Club operator	Admin web panel	Start/stop robot, watch live sessions, view stats, fix wrong guesses
Developer (club member)	Backend, firmware	Logs, configuration, model and prompt updates

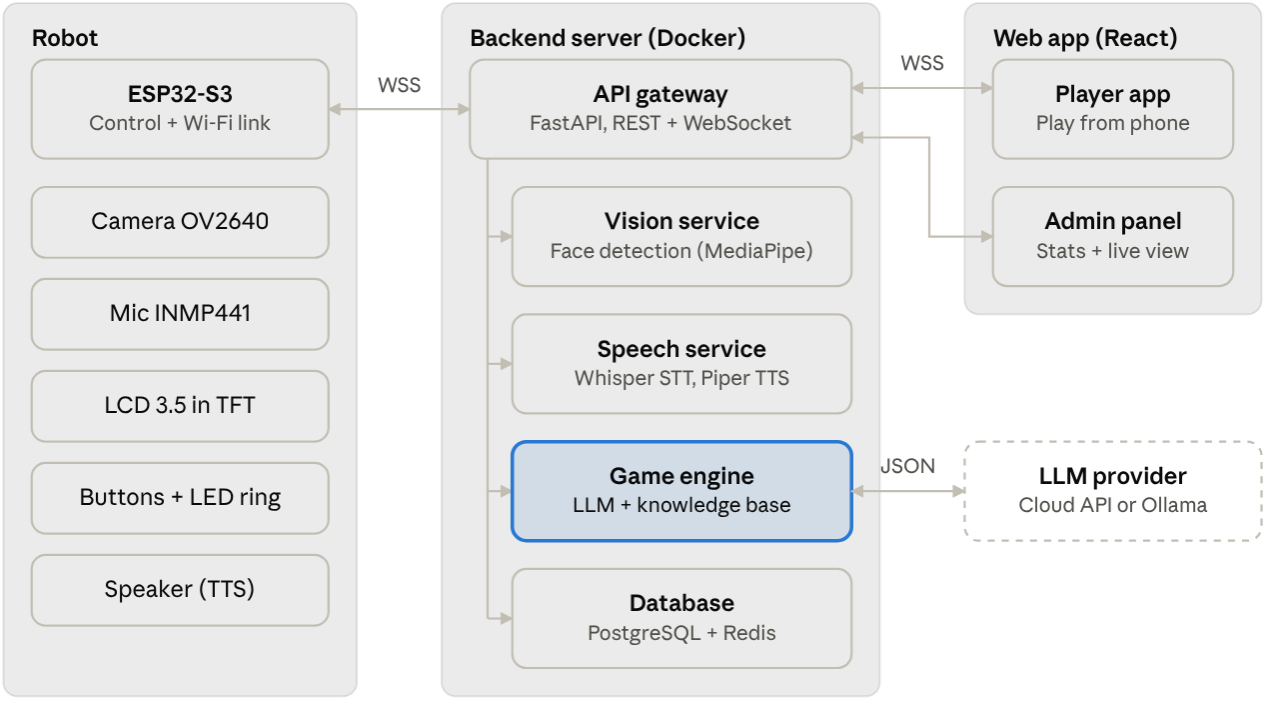
Main use cases

1. **Play on the robot:** the camera detects a face, the robot greets the player, asks questions on the LCD and by voice, listens to answers (Yes / No / Don't know / Probably / Probably not), then shows its guess.
2. **Play on the web:** same game in a browser, answers by buttons; webcam optional.
3. **Teach the robot:** after a wrong guess, the player types the right answer; it is saved to improve future games.
4. **Supervise:** the operator sees robot status (online, camera, mic), active games and win rate.
5. **Event mode:** an attract screen loops when nobody is in front of the robot and wakes up when a face appears.

3. System architecture

The robot is a thin client: it captures video and audio and shows results, while all heavy processing runs on the backend.

Robot and web share one backend that sees, listens and decides



Robot and web call the same API; only the game engine talks to the LLM.

system architecture · robot, backend, web

The robot and web app both connect to one API gateway over secure WebSocket; the gateway routes frames to vision, audio to speech, and answers to the game engine, which is the only part that calls the LLM.

4. Functional requirements

Requirements are tagged **M** (must, v1), **S** (should) or **C** (could, later).

Robot (embedded side)

ID	Requirement	Priority
R1	Stream camera frames to the backend (or run light face detection locally)	M
R2	Record player voice after a "listening" cue and send audio to the backend	M
R3	Display question, answer options, progress and the final guess on the LCD	M
R4	Accept answers by physical buttons as a fallback to voice	M
R5	Speak questions through a speaker (text-to-speech)	S
R6	Show robot "emotions" (thinking, happy, surprised) via LCD animation or LEDs	S
R7	Move head/eyes toward the player with servos	C
R8	Reconnect automatically to Wi-Fi and backend after a drop	M

Backend

ID	Requirement	Priority
B1	Detect faces in frames, return bounding boxes and "player present" events	M
B2	Convert speech to text and map it to one of the 5 answers	M
B3	Generate the next yes/no question from game history using an LLM	M
B4	Decide when to guess (confidence threshold or max 20 questions)	M
B5	Store sessions, questions, answers and outcomes	M

ID	Requirement	Priority
B6	Learn from wrong guesses (save the correct answer and its Q&A trail)	S
B7	Estimate age range / emotion for fun reactions (no identity stored)	C
B8	Expose the same game API to the robot and the web app	M

Web application

ID	Requirement	Priority
W1	Play a full game in the browser with answer buttons	M
W2	Optional webcam and microphone input with clear permission prompts	S
W3	Admin panel: robot status, live session view, statistics, game history	M
W4	Admin login with role-based access	M
W5	Leaderboard of "robot wins vs player wins"	C
W6	Arabic, French and English interface	S

5. Hardware specifications

The recommended main board is an **ESP32-S3** (Wi-Fi, camera interface, I2S microphone support, enough RAM for streaming); a **Raspberry Pi 4/5** is the alternative if the club wants local face detection or a bigger screen.

Component	Recommended part	Role	Notes
Microcontroller	ESP32-S3 with 8 MB PSRAM (e.g. ESP32-S3-EYE / Freenove ESP32-S3 CAM)	Control, Wi-Fi, streaming	Alternative: Raspberry Pi 4 (4 GB)
Camera	OV2640 or OV5640 module	Face detection input	640×480 at 10–15 fps is enough

Component	Recommended part	Role	Notes
Microphone	INMP441 I2S MEMS mic	Voice answers	Mount away from the speaker to limit echo
Display	3.5″ SPI TFT (ILI9488) or 2.8″ touch (ILI9341)	Questions, guess, animations	16×2 character LCD is too small for this UX
Speaker + amp	MAX98357A I2S amp + 3 W speaker	Text-to-speech output	S priority
Buttons	5 arcade buttons (Yes, No, Don't know, Probably, Probably not)	Fallback input	Color-coded to match the UI
Indicators	WS2812 RGB LED ring	Listening / thinking state	
Servos (optional)	2× SG90 or MG90S	Head pan/tilt	C priority
Power	5 V 3 A supply, or 2× 18650 + BMS + 5 V buck	Power all parts	Servos on a separate 5 V rail
Enclosure	3D-printed or laser-cut MDF body	Robot "character"	Leave vents and cable access

Backend server: a laptop or PC on the club network with an NVIDIA GPU (≥ 6 GB VRAM) if running models locally; otherwise any PC/VPS calling cloud APIs.

6. Backend: computer vision and LLM game engine

The backend has three services — vision, speech and game engine — behind one API gateway.

6.1 Computer vision service

- Face detection with **MediaPipe Face Detection** or **YOLOv8-face** (OpenCV for pre-processing).
- Output: bounding boxes, number of faces, "player arrived / player left" events (face present > 1 s / absent > 5 s).
- When several faces are visible, the largest (closest) face is the active player.
- Optional fun features (C): smile detection, age range, emotion — computed live, never stored.
- Target: under 100 ms per frame on GPU, 5–10 fps processed.

6.2 Speech service

- Speech-to-text with **Whisper** (small/medium model, local) or a cloud STT API; must handle Arabic (Darija), French and English.
- Answer mapping: transcribed text → one of 5 answers ("oui", "yes", "na3am", "wah" → Yes). Unclear → ask the player to repeat or press a button.
- Text-to-speech (S): Piper TTS (local) or a cloud TTS voice.

6.3 Game engine (LLM)

The engine is hybrid: a **knowledge base** of characters/objects with known attributes narrows candidates, and the **LLM** writes natural questions and handles characters outside the base.

1. Start session: category (character, animal, object) and language.
2. Each turn, send the LLM: Q&A history, top remaining candidates, question number.
3. LLM returns strict JSON: `{"action": "ask" | "guess", "question": "...", "guess": "...", "confidence": 0.0-1.0}`.
4. Guess when confidence ⩾ 0.85, or forced at question 20.
5. If wrong: continue up to 2 more guesses, then ask the player for the answer and save it (B6).

- LLM options: cloud API (Claude, GPT, Gemini) for best quality, or a local model via **Ollama** (Llama 3.1 8B, Qwen 2.5 7B) for low cost/offline demos.
- Guardrails: system prompt limits topics to family-friendly content; output validated against the JSON schema; retry once on invalid output.
- Target latency: next question displayed within 2 s of the answer.

6.4 Data model (main tables)

Table	Key fields
sessions	id, source (robot/web), language, start/end time, result
turns	session_id, question, answer, confidence
entities	name, category, attributes (JSON), times guessed
devices	robot id, status, last heartbeat, firmware version
users	admin accounts, role

7. UI/UX requirements

The robot and web share one visual identity: a friendly "genie" character, the same 5 answer colors, and the same screen flow.

7.1 Shared screen flow

1. **Idle / attract:** character animation, "Stand in front of me to play!"
2. **Welcome:** player detected, greeting, choose language and category.
3. **Question:** question text (large), question number (e.g. 7/20), 5 answer buttons, listening indicator.
4. **Thinking:** short animation while the LLM works (hides latency).
5. **Guess:** "Are you thinking of… ?" with name (and image if available), Right / Wrong.
6. **Result:** win/lose animation; on a loss, "Teach me" input.
7. **Thank you:** return to idle after 10 s without a face.

7.2 Robot UI (LCD)

- Font ≥ 24 px for questions; maximum 2 lines; readable from 1.5 m.
- Answer colors match the physical buttons: Yes green, No red, Don't know grey, Probably light green, Probably not orange.
- Clear states: listening (LED ring blue), thinking (spinning), speaking (mouth animation).
- Every voice action has a visual equivalent (accessibility for noisy rooms).

7.3 Web UI

- Responsive: phone first, then tablet and desktop.
- Player app: same flow as the robot, large tap targets (≥ 48 px).
- Admin panel: dashboard (robots online, games today, win rate), live session view (camera feed with face boxes, current Q&A), history and entity editor.
- Right-to-left layout support for Arabic.
- Light and dark themes.

7.4 UX deliverables

- [ ] User personas and journey map
- [ ] Low-fidelity wireframes (Figma)
- [ ] Character design and animation set
- [ ] High-fidelity mockups for robot LCD, web player, admin panel
- [ ] Usability test with 5+ club visitors before final build

8. Non-functional requirements

Area	Requirement
Performance	Next question in ⩽ 2 s; face detected in ⩽ 1 s after a player arrives; STT result in ⩽ 1.5 s
Reliability	Robot runs 4 h continuously at an event; auto-reconnect within 10 s
Accuracy	Robot guesses correctly in ⩾ 60% of games within 20 questions (v1 target)
Privacy	No face images or voice recordings stored by default; only anonymous session data; visible notice "camera in use"
Security	HTTPS/WSS between robot, web and backend; device token per robot; admin login with hashed passwords
Safety	Family-friendly LLM output; no exposed wires or sharp edges; power supply fused
Maintainability	Code on Git with README per module; configuration in env files; prompts versioned
Portability	Backend runs in Docker; switching LLM provider requires only a config change
Languages	Arabic, French, English for text and voice

9. Tech stack

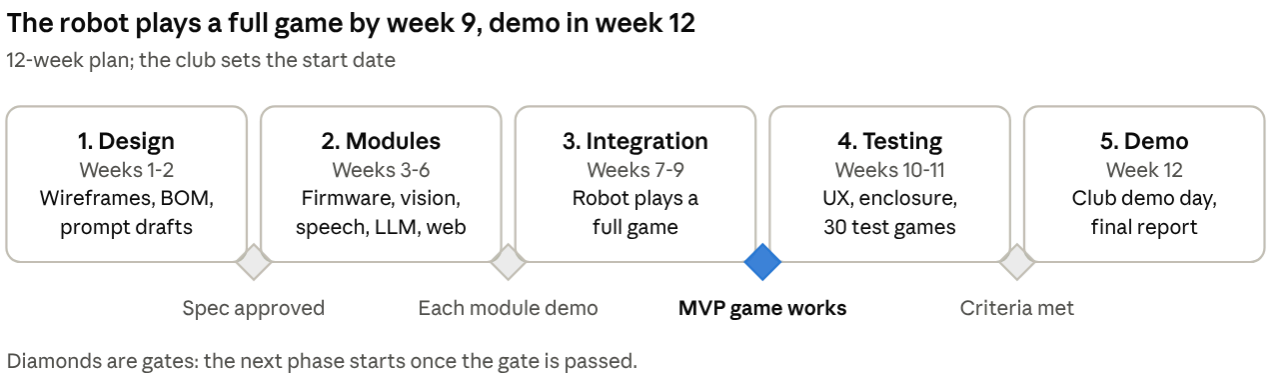
Layer	Technology
Firmware	C++ with Arduino-ESP32 or ESP-IDF; LVGL for LCD graphics; WebSocket client
Communication	WebSocket (real-time game + audio), HTTP/REST (config, stats), MQTT optional for device status
Backend API	Python 3.11 + FastAPI
Computer vision	OpenCV, MediaPipe or Ultralytics YOLOv8-face
Speech	Whisper / faster-whisper (STT), Piper (TTS)

Layer	Technology
LLM	Cloud API or Ollama (local), with LangChain or plain SDK calls
Database	PostgreSQL (or SQLite for the prototype); Redis for live sessions
Web front end	React + Vite, Tailwind CSS, i18next (AR/FR/EN)
Design	Figma
DevOps	Git + GitHub, Docker Compose, GitHub Actions for tests

10. Team roles and planning

A team of 7–10 members can deliver v1 in about 12 weeks, working in five parallel workstreams.

Role	Members	Responsibilities
Project lead	1	Planning, weekly meetings, integration decisions, Git repo
Embedded / hardware	2	Wiring, firmware, LCD UI (LVGL), power, enclosure
Computer vision / AI	2	Face detection, speech-to-text, LLM prompts and game engine
Backend	1–2	API, WebSocket, database, Docker deployment
Web front end	1–2	Player web app, admin panel
UI/UX designer	1	Character, wireframes, mockups, usability tests



project roadmap · 5 phases, 4 gates, 12 weeks

Week 9 is the key checkpoint: if the full game does not run end to end on the robot, the team cuts S and C features to protect the demo.

11. Budget estimate

The robot hardware costs roughly **23,000–37,000 DZD** with the ESP32-S3 option; prices are approximate and must be checked with local suppliers.

Item	Qty	Approx. cost (DZD)
ESP32-S3 camera board (8 MB PSRAM)	1	3,500–5,000
INMP441 microphone	1	800–1,200
3.5″ TFT display	1	4,000–6,000
MAX98357A amp + speaker	1	1,300–2,000
Arcade buttons	5	1,500–2,500
WS2812 LED ring	1	800–1,500
Servos SG90/MG90S (optional)	2	1,200–2,000
Power (supply or 18650 + BMS + buck)	1 set	2,500–4,000
Enclosure (3D print / MDF)	1	4,000–8,000
Wires, PCB, connectors, spare parts	—	3,000–5,000
Total robot (ESP32 option)		**≈ 22,600–37,200**

- Raspberry Pi 4 option: add about 15,000–20,000 DZD.
- LLM cloud API: about 5–20 USD/month at club usage; 0 with a local model on an existing GPU PC.

- Hosting: 0 on a club PC; about 5–10 USD/month for a small VPS if the web version must be public.

12. Risks, acceptance criteria and deliverables

Risks and mitigations

Risk	Impact	Mitigation
Noisy room makes voice answers fail	High	Physical buttons always available; push-to-talk; directional mic
LLM too slow or internet drops	High	"Thinking" animation; local Ollama fallback; knowledge-base-only mode
Darija speech poorly recognized	Medium	Keyword list per language; button fallback; test with real club members
ESP32 memory too small for audio + video + UI	Medium	Lower frame rate, stream only when needed, or switch to Raspberry Pi
LLM gives inappropriate or invalid output	Medium	Strict system prompt, JSON schema validation, content filter
Team availability (exams)	Medium	Clear module owners, MVP first, weekly check-ins
Privacy concerns from visitors	Low	No storage of images/audio, visible notice, camera-off option

Acceptance criteria (v1)

- [ ] A visitor can play a full game on the robot using voice or buttons without help.
- [ ] Face detection starts a game automatically in ⩽ 1 s.
- [ ] Next question appears in ⩽ 2 s in 90% of turns.
- [ ] Correct guess in ⩾ 60% of 30 test games.
- [ ] Web version plays the same game from a phone browser.
- [ ] Admin panel shows robot status and game history.
- [ ] Robot runs 4 hours at a club event without restart.

Deliverables

- Working robot prototype and enclosure
- Source code (firmware, backend, web) on GitHub with documentation
- Figma design files and character assets

- Wiring diagram and bill of materials
- Test report and demo video
- Final presentation for the club